算法设计与分析（第七讲）

黄 橙
计算与智能创新学院
chuang@fudan.edu.cn

课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

求解复杂算法问题

线性规划，在线算法，AI算法

理解高效计算的边界

P vs NP，NP 完全性，近似算法

动态规划与贪心算法

动态规划



比较多个选择，
复用子问题的最优解。

贪心算法



每一步选择当前最优，
不撤销已作出的选择。

找零问题

结账时需要找零 6 元，可用的硬币面额为 1 元、2 元、5 元。
每种硬币数量充足。

目标：恰好凑出找零金额，并使硬币总数最少。

贪心选择：每次取不超过剩余金额的最大面额。

$$6 = 5 + 1$$

共 2 枚硬币，达到最优。

找零问题

如果可用硬币换成 1 元、3 元、4 元呢？

6 元怎么找？

仍然每次取不超过剩余金额的最大面额，
能保证使用的硬币最少吗？

找零问题

用 1 元、3 元、4 元硬币，凑出 6 元。

每次取最大面额	$6 = 4 + 1 + 1$	3 枚
最优方案	$6 = 3 + 3$	2 枚

同样是找零，为什么这次贪心选择没有得到最优解？

是否最优

关键：当前的贪心选择，是否仍能扩展为一个全局最优解？

$\{1, 2, 5\}$　　找零 6 元时，先取 5 元仍能得到最优解。

$\{1, 3, 4\}$　　找零 6 元时，先取 4 元会排除最优方案。

因此，需要证明贪心选择的正确性，
不能只凭“眼前最优”判断整个方案最优。

局部规则与全局目标

先写清楚可行解与目标函数，再定义每一步的优先级。

层次	找零问题中的含义
可行解	硬币面额之和等于待找金额
全局目标	使硬币枚数最少
局部规则	优先选不超过剩余金额的最大面额

成功实例只能确认这次结果；一个反例就能否定普遍最优的断言。

贪心证明的责任：每一步之后，仍有一个最优解容纳当前选择。

活动选择问题

输入：活动集合 $S = \{1, 2, \ldots, n\}$

活动 i 的开始时间和结束时间分别为 s_i, f_i （开始早于结束）。

目标：选择活动数最多的两两相容活动集 $A \subseteq S$

相容条件（允许首尾相接）：$s_i \geq f_j$ 或 $s_j \geq f_i$

输入实例

i	1	2	3	4	5	6	7	8	9	10
s_i	1	3	2	5	4	5	6	8	8	2
f_i	4	5	6	7	9	9	10	11	12	13

贪心算法的设计思想

按某个指标排序并重新编号，
依次加入与所有已选活动相容的活动。

策略 1：开始时间早的优先

$$s_1 \leq s_2 \leq \ldots \leq s_n$$

策略 2：占用时间少的优先

$$f_1 - s_1 \leq f_2 - s_2 \leq \ldots \leq f_n - s_n$$

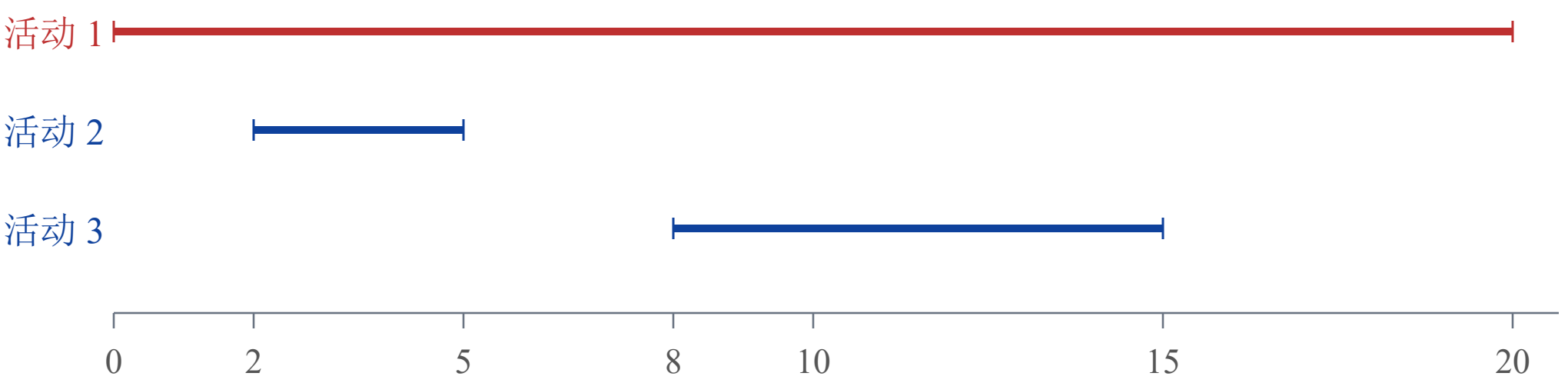
策略 3：结束时间早的优先

$$f_1 \leq f_2 \leq \ldots \leq f_n$$

策略 1 的反例

策略：开始时间早的优先 $S = \{1, 2, 3\}$

$s_1 = 0,\quad f_1 = 20 \qquad s_2 = 2,\quad f_2 = 5 \qquad s_3 = 8,\quad f_3 = 15$

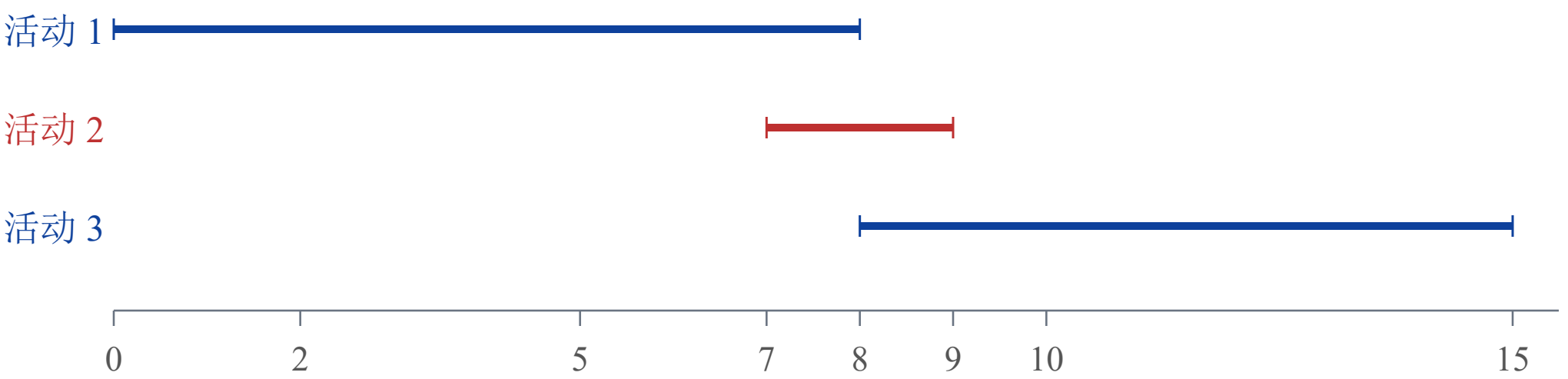


该策略只能选活动 1，而活动 2、3 相容，可以选 2 项。

策略 2 的反例

策略：占用时间少的优先 $S = \{1, 2, 3\}$

$s_1 = 0,\quad f_1 = 8 \qquad s_2 = 7,\quad f_2 = 9 \qquad s_3 = 8,\quad f_3 = 15$



该策略只能选活动 2，而活动 1、3 相容，可以选 2 项。

贪心算法的设计思想

挑选过程包含多步判断。每一步按某个局部优先规则选择活动，
新选活动必须与所有已选活动相容。

最优策略：结束时间早的优先

$$f_1 \leq f_2 \leq \cdots \leq f_n$$

按结束时间非降序排序并重新编号，再从前向后依次挑选。

贪心算法伪码

算法 Greedy Select

输入：活动集合 S，开始、结束时间 s_i, f_i，且 $f_1 \le \cdots \le f_n$。
输出：选中的活动子集 $A \subseteq S$。

1. $n \leftarrow |S|$
2. if $n = 0$ then return $\emptyset$
3. $A \leftarrow \{1\}$，$j \leftarrow 1$
4. for $i \leftarrow 2$ to n do
5. 　if $s_i \ge f_j$ then
6. 　　$A \leftarrow A \cup \{i\}$
7. 　　$j \leftarrow i$
8. return A

其中 j 为最后选入活动的编号。非空解的最后完成时间为

$$t = \max\{f_k : k \in A\}$$

运行实例

扫描活动 10：跳过

输入：另一个输入实例，$S = \{1,2,3,\ldots,10\}$。

i	**1**	2	3	**4**	5	6	7	**8**	9	10
s_i	**1**	3	0	**5**	3	5	6	**8**	8	2
f_i	**4**	5	6	**7**	8	9	10	**11**	12	13

扫描活动 10：$s_{10} = 2 < f_8 = 11$，跳过。依次选择活动 1、4、8。

$$A = \{1,4,8\}, \qquad j = 8, \qquad t = 11$$

时间复杂度：$O(n \log n) + O(n) = O(n \log n)$（排序加扫描）。

如何证明该算法对所有实例都能得到最优解？

分析

算法：将活动按结束时间非降序排序，先选第一个活动。
之后每次选择与最后已选活动相容、且结束时间最早的活动。

交换论证：若某个最优解的第一个活动不是贪心所选活动，
就用贪心选择替换它。替换后的活动结束时间不晚于原活动，
因此不会减少后续可安排的活动数。

重复这个过程，可以得到与最优解同样大小的贪心解。
所以，策略3能够得到最优解。

交换论证的证明责任

在当前剩余实例中，贪心选择最早结束的活动 g。
若最优解 O 不含 g，取 O 中最先结束的活动 j。

$$O' = (O - \{j\}) \cup \{g\}$$

检查项	活动选择中的依据
替换后仍可行	新活动结束时间不晚于旧活动，后续安排仍可接上
目标值不变	只替换一个活动，已选活动数量不变

由此得到一个包含贪心选择的最优解；最优解可以不唯一。

再对剩余实例使用同样的性质，归纳连接每一次安全选择。

贪心算法的特点

贪心法常用于组合优化问题。

求解过程包含多步选择。策略满足正确性条件时，
最终的选择序列对应于问题的最优解。

每一步依据局部优先规则作出选择，
该规则是否满足贪心选择性质，决定了算法能否保证最优。

需要分析算法，并证明贪心策略的正确性。
证明某个贪心策略不正确，可以举反例。

常见优点：算法结构简单，时间与空间开销往往较低。

贪心法的正确性证明

数学归纳法

数学归纳法

例：证明对于任意正整数 $n \geq 1$，都有

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

归纳基础：$n = 1$，左边为 1，右边为 $1(1+1)/2 = 1$。
归纳假设：等式对 n 成立。证明 $n+1$ 的情形：

$$\begin{aligned} & 1 + 2 + \cdots + n + (n+1) \\ = & (1 + 2 + \cdots + n) + (n+1) \\ = & \frac{n(n+1)}{2} + (n+1) \\ = & \frac{(n+1)(n+2)}{2} \end{aligned}$$

数学归纳法

数学归纳法用于证明涉及自然数的命题 $P(n)$。

归纳基础：证明 $P(1)$ 为真。

归纳步骤：对任意固定的 $n \geq 1$，假设 $P(n)$ 为真，
证明 $P(n+1)$ 为真。

$$\forall n \geq 1, P(n) \Rightarrow P(n+1)$$

$$P(1)$$

$$P(1) \Rightarrow P(2) \Rightarrow P(3) \Rightarrow \cdots$$

数学归纳法变形

强归纳法用于证明定义在正整数上的命题 $P(n)$。

归纳基础：证明 $P(1)$ 为真。

归纳步骤：对任意 $n \geq 2$，假设对所有 $1 \leq k < n$，
命题 $P(k)$ 都成立，证明 $P(n)$ 也成立。

$$P(1)$$

$$P(1) \Rightarrow P(2)$$

$$(P(1) \wedge P(2)) \Rightarrow P(3)$$

$$\dots$$

区别

两种方法都先证明归纳基础 $P(1)$，归纳假设的形式不同。

普通归纳法：由当前命题推出下一命题。

$$P(1) \Rightarrow P(2) \Rightarrow P(3) \Rightarrow \cdots$$

强归纳法：同时利用所有前面的命题。

$$\left.\begin{matrix} P(1) \\ P(1) \to P(2) \end{matrix}\right\} \to \left.\begin{matrix} P(1) \\ P(2) \\ P(3) \end{matrix}\right\} \to P(4) \quad \ldots$$

例子

命题 $P(n)$：任意整数 $n \geq 2$ 都能表示成若干个质数的乘积。

尝试直接使用普通归纳：

基础：$2 = 2$，命题成立。
步骤：假设 $P(n)$ 成立，希望证明 $P(n+1)$。

困难：$n+1$ 的因子不一定与 n 有直接联系。
例如 $4 = 2\times2$，并不能直接利用 3 的分解。

仅使用 $P(n)$ 不够方便，强归纳的表述更直接。
两种归纳法在逻辑上等价，并非普通归纳不能证明本命题。

例子

命题 $P(n)$：任意整数 $n \geq 2$ 都能表示成若干个质数的乘积。

用强归纳法：基础情形 $n = 2$ 已成立。

假设：对所有整数 k，当 $2 \leq k \leq n$ 时，$P(k)$ 都成立。

证明 $P(n+1)$：
若 $n+1$ 是质数，命题直接成立。
否则，可写为 $n+1 = ab$，其中 $2 \leq a, b < n+1$。

根据归纳假设，a 和 b 都能分解为质数乘积，
因此 $n+1$ 也能分解为质数乘积。

关系

普通归纳：假设 $P(n)$ 成立，证明 $P(n+1)$ 成立。

$$P(n) \Rightarrow P(n+1)$$

强归纳：假设 $P(1), \ldots, P(n)$ 都成立，证明 $P(n+1)$ 成立。

$$(P(1) \wedge P(2) \wedge \cdots \wedge P(n)) \Rightarrow P(n+1)$$

两种方法逻辑等价，区别在归纳假设的形式和使用方式。

贪心算法正确性证明

证明步骤

建立一个与自然数 n 有关的命题，
断言该贪心策略最终能够得到最优解。

其中 n 可以表示算法执行的步数，也可以表示问题规模。

证明该命题对所有适用的 n 成立：
归纳基础：从最小规模或初始步骤开始。
归纳步骤：使用普通归纳或强归纳。

归纳命题要覆盖剩余状态

归纳命题要包含算法运行后仍会变化的参数。

以稍后讲到的装载问题为例：若 $w_1 \le C$ 并装入箱子 1，
箱子数减少，剩余容量也改变。

$$(n, C) \rightarrow (n-1, C-w_1)$$

因此，归纳假设应覆盖任意合法重量、任意 $C \ge 0$。
只对某个固定容量的断言，无法直接用于新的剩余容量。

活动选择同样需要记录“已选前缀”及其最后结束时间，
确保归纳假设覆盖接下来真正处理的剩余活动。

活动选择算法

给定活动集合 S，每个活动 i 对应半开区间 $[s_i, f_i)$。

按结束时间非降序排序并重新编号：

$$f_1 \leq f_2 \leq \cdots \leq f_n$$

贪心策略：反复选择与已选集合相容、
且结束时间最早的活动。记所得活动序列为

$$i_1, i_2, \ldots, i_m$$

归纳不变式（要证明的命题）

要证明的归纳不变式：

对任意 $k \in \{1, \ldots, m\}$，存在一个最优解 A_k，将其中的活动按结束时间非递减顺序排列后，前 k 个活动依次为 $i_1, i_2, \ldots, i_k$

归纳证明：归纳基础

第三步：相容性保留，活动数不变

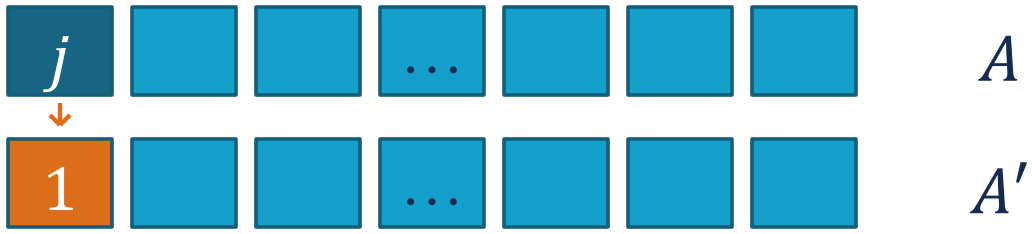
令 $S = \{1,2,\dots,n\}$，且 $f_1 \le \cdots \le f_n$。
归纳基础：$k = 1$，证明存在包含活动 1 的最优解。

任取最优解 A，按结束时间非降序排列。若首项为 $j \neq 1$，用活动 1 替换它，得到

$$A' = (A - \{j\}) \cup \{1\}$$

因为 $f_1 \le f_j$，后续活动仍然相容，且活动数不变。
因此 A' 仍是最优解，并且包含活动 1。

$$f_1 \le f_j$$



归纳步骤：由 k推 $k+1$

假设 $k < m$，存在最优解 A_k，其前 k 个活动为 $(i_1, \dots, i_k)$。
因为最优解大小至少为 m，它有第 $k+1$ 个活动，记为 j。

贪心选择满足 $s_t \geq f_{i_k}$ 且结束时间最早的活动 i_{k+1}：

$$f_{i_{k+1}} = \min\{f_t : t \in S, s_t \geq f_{i_k}\} \leq f_j$$

将 A_k 中的 j 替换为 i_{k+1}，得到 A_{k+1}。
由 $s_{i_{k+1}} \geq f_{i_k}$，新活动与前缀相容。
由 $f_{i_{k+1}} \leq f_j$，后续活动仍可按原顺序安排。

因此，A_{k+1} 仍然可行，活动数与 A_k 相同，仍是最优解。
它的前 $k+1$ 个活动恰为 $(i_1, \dots, i_k, i_{k+1})$。
归纳步骤成立。

数学归纳法变形

命题 $P(k)$：存在一个最优解，其前 k 个活动等于
贪心前缀 $(i_1, \ldots, i_k)$。

归纳基础：由前述首项交换论证，$P(1)$ 成立。

强归纳形式：假设 $P(1), \ldots, P(k)$ 均成立，证明 $P(k+1)$。
实际上，前页的交换论证只需要 $P(k)$，普通归纳已足够。

当 $k = m$ 时，某个最优解以完整贪心解为前缀。
若它还有后续活动，贪心算法就不会终止。
因此这个最优解恰有 m 项，贪心解最优。

归纳假设的“信息量”不同

对比项	普通归纳法	强归纳法
归纳假设	只假设 $P(k)$ 成立	假设 $P(1), \ldots, P(k)$ 都成立
推理依据	由 $P(k)$ 推出 $P(k+1)$	可使用所有前面命题的信息，推出 $P(k+1)$
形式区别	单步依赖	可综合前面多个情形

普通归纳像爬楼梯：证明能从第 k 级上到第 $k+1$ 级。
强归纳像搭桥：到第 $k+1$ 个岛时，可利用前面所有岛的资源。

两种归纳法在逻辑上等价，区别是归纳假设的表述方式。

用动态规划方式求解

将原问题分成子问题：在活动 i 结束后、活动 j 开始前，
最多能安排多少项活动？

$$S_{ij} = \{k: f_i \leq s_k < f_k \leq s_j\}$$

定义状态：$c[i,j]$ 表示从 S_{ij} 中选择的最大相容活动数，
不包含边界活动 i、j。

边界条件：若 $S_{ij} = \emptyset$，则 $c[i,j] = 0$。

引入虚拟边界：$f_0 = -\infty$，$s_{n+1} = +\infty$。
原问题的答案为 $c[0, n+1]$。

用动态规划方式求解

用 $c[i,j]$ 表示集合 S_{ij} 的最优解大小。
固定选入活动 k 时，候选值为 $c[i,k]+c[k,j]+1$。

若 $S_{ij}=\emptyset$：

$$c[i,j]=0$$

若 $S_{ij}\neq\emptyset$，对所有可选活动取最大值：

$$c[i,j]=\max_{k\in S_{ij}}\{c[i,k]+c[k,j]+1\}$$

状态数为 $O(n^2)$，每个状态最多枚举 $O(n)$ 个活动。
时间复杂度为 $O(n^3)$，空间复杂度为 $O(n^2)$。

用动态规划方式求解

方法	思路	时间复杂度	备注
贪心算法	每次选结束最早的 相容活动	$O(n \log n)$ （含排序）	简单高效
动态规划 （区间DP）	枚举边界 (i, j)， 利用最优子结构递推	$O(n^3)$	可扩展到 加权版本

这里的复杂度对应上述区间DP实现，并不代表所有DP实现。

活动带权后，选择依据发生变化

活动 j 的价值为 v_j；目标由活动数改为价值和。
按结束时间排序，令 $F(j)$ 为前 j 项活动能取得的最大价值。

活动	区间	价值
a	$[0,2)$	2
b	$[2,4)$	2
c	$[0,4)$	9

教学实例：最早结束选 a,b，价值为 4；只选 c，价值为 9。

$$F(j) = \max\{F(j-1), v_j + F(p(j))\}, \qquad F(0) = 0$$

$p(j)$ 为 j 之前最后一个相容活动的编号；无前驱时 $p(j) = 0$。
排序及二分求前驱耗时 $O(n \log n)$，递推耗时 $O(n)$。

两个贪心算法的应用

最优装载问题

最小延迟调度

最优装载问题

有 n 个集装箱，编号为 $1,2,\ldots,n$。箱子 i 的重量为 $w_i > 0$，
轮船的载重上限为 $C \geq 0$，无体积限制。
怎样装载，才能使上船的集装箱数量最多？

若某个箱子满足 $w_i > C$，它不可能装入，可以预先排除。

这是0-1背包问题的一个特例：
集装箱对应物品，重量为 w_i，每件物品的价值均为 $v_i = 1$。
轮船载重上限 C 对应背包容量 b。

建模

用向量 $(x_1, x_2, \dots, x_n)$ 表示装载方案，$x_i \in \{0,1\}$。
$x_i = 1$ 当且仅当第 i 个集装箱装上船。

目标函数：

$$\max \sum_{i=1}^{n} x_i$$

约束条件：

$$\sum_{i=1}^{n} w_i\, x_i \le C, \qquad x_i \in \{0,1\}, \qquad i = 1,2,\dots,n$$

算法设计

贪心策略：轻者优先。

将集装箱按重量非降序排序并重新编号：

$$w_1 \leq w_2 \leq \cdots \leq w_n$$

按编号从小到大依次装箱。
直到再装入下一个箱子会使总重量超过载重上限时停止。

正确性证明

命题：对任意规模为 n 的装载实例和任意非负容量，
轻者优先的贪心算法都得到最优解。

将箱子从轻到重编号为 $1,2,\ldots,n$。

归纳基础：证明只含 1 个箱子的实例中，贪心解最优。

归纳步骤：假设对任意 n 个箱子的实例，贪心法都最优，
证明对任意 $n+1$ 个箱子的实例也成立。

归纳基础（$n = 1$）

只有 1 个箱子时：

若 $w_1 \le C$，贪心算法装入该箱子，选中 1 个。

若 $w_1 > C$，任何方案都无法装入，选中 0 个。

两种情况下都达到最优，因此 $P(1)$ 成立。

归纳步骤：对 $n+1$ 个箱子仍成立

设有 $n+1$ 个箱子，且 $w_1 \le w_2 \le \cdots \le w_{n+1}$。
记贪心解为 G_{n+1}，最优装载集合为 O_{n+1}。

若 $w_1 > C$，则所有箱子都不可装入，空集最优。
以下设 $w_1 \le C$，按最优解是否包含箱子 1 分类。

情况1：最优解 O_{n+1} 包含箱子 1。

去掉箱子 1 后，剩余 n 个箱子的容量为 $C - w_1$。
原最优解的剩余部分必为该子问题的最优解，
否则可改进原方案，与其最优性矛盾。

根据归纳假设 $P(n)$，贪心算法在子问题中最优，故整体最优。

归纳步骤：对 $n+1$ 个箱子仍成立

情况2：最优解 O_{n+1} 没有装入箱子1

若 $w_1 > C$，所有箱子都装不下，空集最优。以下设 $w_1 \le C$。

此时最优解非空。任选其中一个箱子 i，都有 $w_i \ge w_1$。

将箱子 i 替换为箱子1，得到新方案

$$O' = (O_{n+1} \setminus \{i\}) \cup \{1\}$$

新方案仍可行：$\sum_{j \in O'} w_j \le \sum_{j \in O_{n+1}} w_j \le C$。
两者箱子数量相同。因此，O' 也是最优解，并且包含箱子1。

归纳步骤：对 $n+1$ 个箱子仍成立

回到包含箱子1的情形

去掉箱子1后，剩余容量为 $C - w_1$，剩余箱子数量为 n。

根据归纳假设 $P(n)$，贪心算法在这个子问题上得到最优解。

再加回箱子1，整体贪心解 G_{n+1} 也是最优解。

证明思路

“轻箱优先”策略最优：

当最轻箱子能够装入时，若某个最优解没有选它，
就可以用它替换一个重量不小于它的箱子。
箱子数量不变，总重量不增加。

因此，总存在一个包含最轻箱子的最优解。
对剩余箱子和剩余容量继续归纳，贪心始终最优。

装载证明依赖“数量最多”

轻者优先的交换：箱子数量不变，总重量不增加，因此保留数量目标。

$$C = 6, \qquad (w_1, w_2, w_3) = (2,3,4)$$

可行方案	箱子数量	装载总重
选重量 2,3（轻者优先）	2	5
选重量 2,4	2	6

若目标改成总重量最大，轻者优先便未达到该目标。

最小延迟调度问题

给定客户集合 A。客户 i 的服务时间为 t_i，截止时间为 d_i，
二者均为正整数。所有客户从时刻0起可用，由一台机器不可抢占地服务。

调度 $f: A \to \mathbb{Z}_{\geq 0}$ 指定各客户的开始时间。定义非负延迟：

$$D_i = \max\{0, f(i) + t_i - d_i\}$$

目标：使所有客户中的最大延迟最小。

$$\min_f \max_{i \in A} D_i$$

单机约束：任意不同客户 $i, j \in A$ 的服务区间不能重叠，即

$$f(i) + t_i \leq f(j) \text{ 或 } f(j) + t_j \leq f(i)$$

实例：调度1

$$A = \{1,2,3,4,5\}, \qquad T = \langle 5,8,4,10,3\rangle, \qquad D = \langle 10,12,15,11,20\rangle$$

调度1：按客户编号顺序安排

$$f_1(1) = 0, f_1(2) = 5, f_1(3) = 13, f_1(4) = 17, f_1(5) = 27$$

按客户编号1—5的延迟：0，1，2，16，10　　**最大延迟：16**



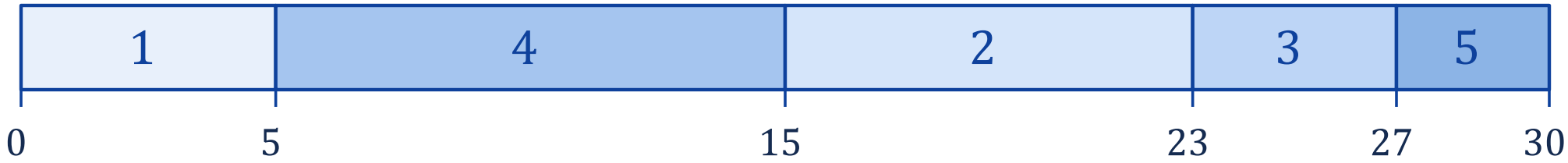
实例：调度2

$$A = \{1,2,3,4,5\}, \qquad T = \langle 5,8,4,10,3\rangle, \qquad D = \langle 10,12,15,11,20\rangle$$

调度2：按截止时间非降序安排

$$f_2(1) = 0, f_2(2) = 15, f_2(3) = 23, f_2(4) = 5, f_2(5) = 27$$

按客户编号1—5的延迟：0，11，12，4，10　　**最大延迟：12**



贪心策略

策略1：按照服务时间 t_i 从小到大排列。

策略2：按照 $d_i - t_i$ 从小到大排列。

策略3：按照截止时间 d_i 从小到大排列。

策略1的反例

$$t_1 = 1, d_1 = 100, t_2 = 10, d_2 = 10$$

先1后2的最大延迟为1；先2后1的最大延迟为0。

策略2的反例

$$t_1 = 1, d_1 = 2, t_2 = 10, d_2 = 10$$

先2后1的最大延迟为9；先1后2的最大延迟为1。

算法伪码

算法 Schedule

输入：客户集合 A、服务时间 T、截止时间 D；$n = |A| \geq 1$
输出：开始时间函数 f

1. 按截止时间排序并重新编号，使 $d_1 \leq d_2 \leq \cdots \leq d_n$
2. $f(1) \leftarrow 0$
3. for $i \leftarrow 2$ to n do
4. $\quad f(i) \leftarrow f(i-1) + t_{i-1}$
5. return f

前一个客户完成后立即开始服务当前客户。
设计思想：按截止时间非降序安排任务，中间不留空闲。

交换论证

假设当前顺序中存在相邻任务 i, j，满足 $d_i > d_j$，
但任务 i 在任务 j 之前执行。

只交换这两个相邻任务：

$i \to j$	$j \to i$
交换前	交换后

比较两种顺序下的最大延迟。

交换相邻逆序 (i, j) 不影响最优性

引理1：交换相邻逆序，不会增加最大延迟。

交换不影响其他客户的延迟，且任务 j 提前完成。

$d_i > d_j$

$f_1(i) = s$　　$f_1(j) = s + t_i$　　$f_1(j) + t_j = s + t_i + t_j$

交换前



交换后



$f_2(j) = s$　　$f_2(i) = s + t_j$

$f_2(i) + t_i = s + t_j + t_i$

最大延迟可能保持不变，也可能减小。

细节

先比较有符号延误量 $L = C - d$，非负延迟为 $D = \max\{0, L\}$。

交换前（顺序 i, j）

$$C_i = s + t_i, \qquad C_j = s + t_i + t_j$$

$$L_i = C_i - d_i = s + t_i - d_i, \qquad L_j = C_j - d_j = s + t_i + t_j - d_j$$

交换后（顺序 j, i）

$$C'_j = s + t_j, \qquad C'_i = s + t_j + t_i$$

$$L'_j = C'_j - d_j = s + t_j - d_j, \qquad L'_i = C'_i - d_i = s + t_j + t_i - d_i$$

细节

（A）比较交换前两个任务的延误量

$$L_j - L_i = (s + t_i + t_j - d_j) - (s + t_i - d_i) = t_j + (d_i - d_j) \geq t_j$$

$$L_j \geq L_i + t_j$$

（B）比较交换后与交换前

$$L'_j = s + t_j - d_j = (s + t_i + t_j - d_j) - t_i = L_j - t_i \leq L_j$$

$$L'_i = s + t_j + t_i - d_i = (s + t_i - d_i) + t_j = L_i + t_j \leq L_j$$

因此 $\max\{L'_i, L'_j\} \leq L_j = \max\{L_i, L_j\}$。
取 $\max\{0, L\}$ 后，最大非负延迟也不增加。

证明要点

从一个没有空闲时间的最优调度出发，
通过交换相邻逆序，得到按截止时间非降序排列的最优调度。

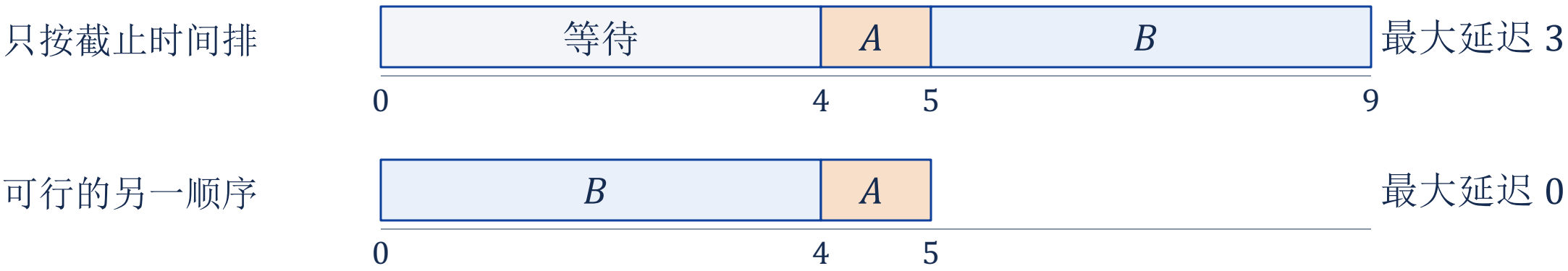
1. 如果排列中存在逆序，就存在相邻逆序。
2. 交换相邻逆序任务，由引理1，最大延迟不增加，因而仍然最优。
3. 每次交换使逆序数减1，至多交换 $n(n-1)/2$ 次，
   就得到一个没有逆序的最优调度。
4. 这正是截止时间优先的排序规则。并列截止时间可按算法规则处理。

因此，截止时间优先的贪心算法得到最优解。

调度定理的建模前提

截止时间优先的证明使用了：所有任务从时刻 0 起可用。

加入释放时间的实例：
$r_A = 4, t_A = 1, d_A = 5$；$r_B = 0, t_B = 4, d_B = 6$。



任务 A 在时刻 4 才可开始。先处理 B，两项任务都能按期完成。

释放时间限制了任务的开始时刻；原先的“消除空闲”和相邻交换须重新论证。

二元前缀码

用0—1字符串表示字符，要求任何字符的码字
都不是其他字符码字的前缀。

非前缀码的例子

$$a: 001, \qquad b: 00, \qquad c: 010, \qquad d: 01$$

字符串0100001存在两种解码：

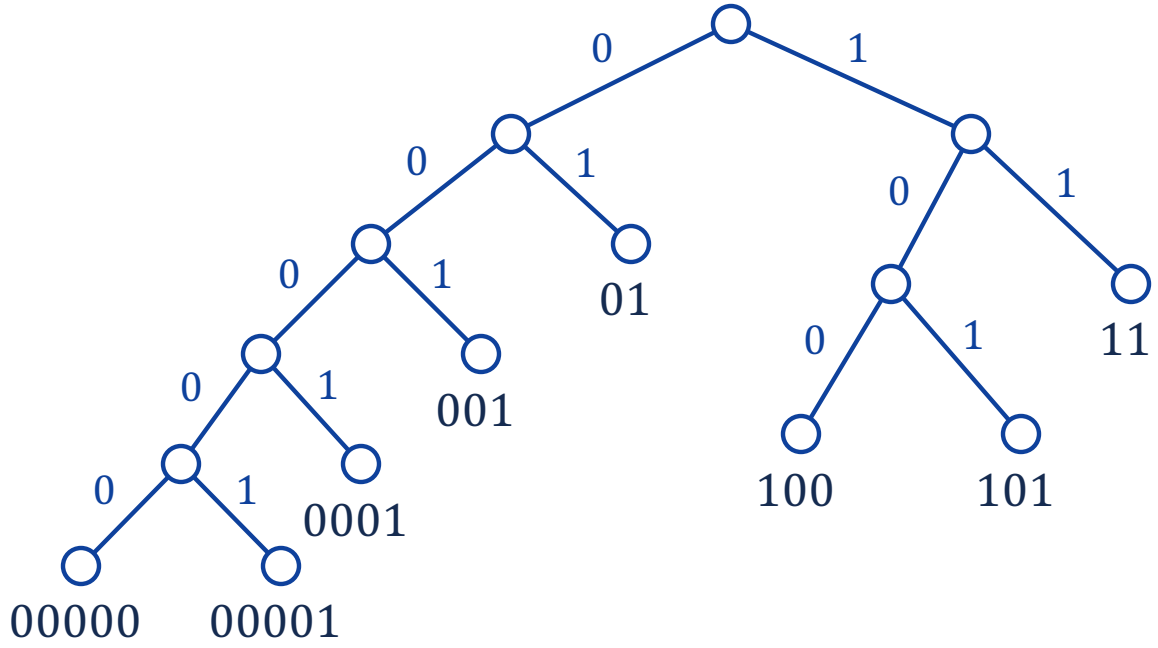
解码1：01，00，001，对应 d，b，a
解码2：010，00，01，对应 c，b，d

前缀码的二叉树表示

前缀码：

{00000,00001,0001,001,01,100,101,11}

构造规则：
0表示走向左子树。
1表示走向右子树。
每个码字对应一个叶子节点。
最长码字长度等于树的最大深度。



平均传输位数

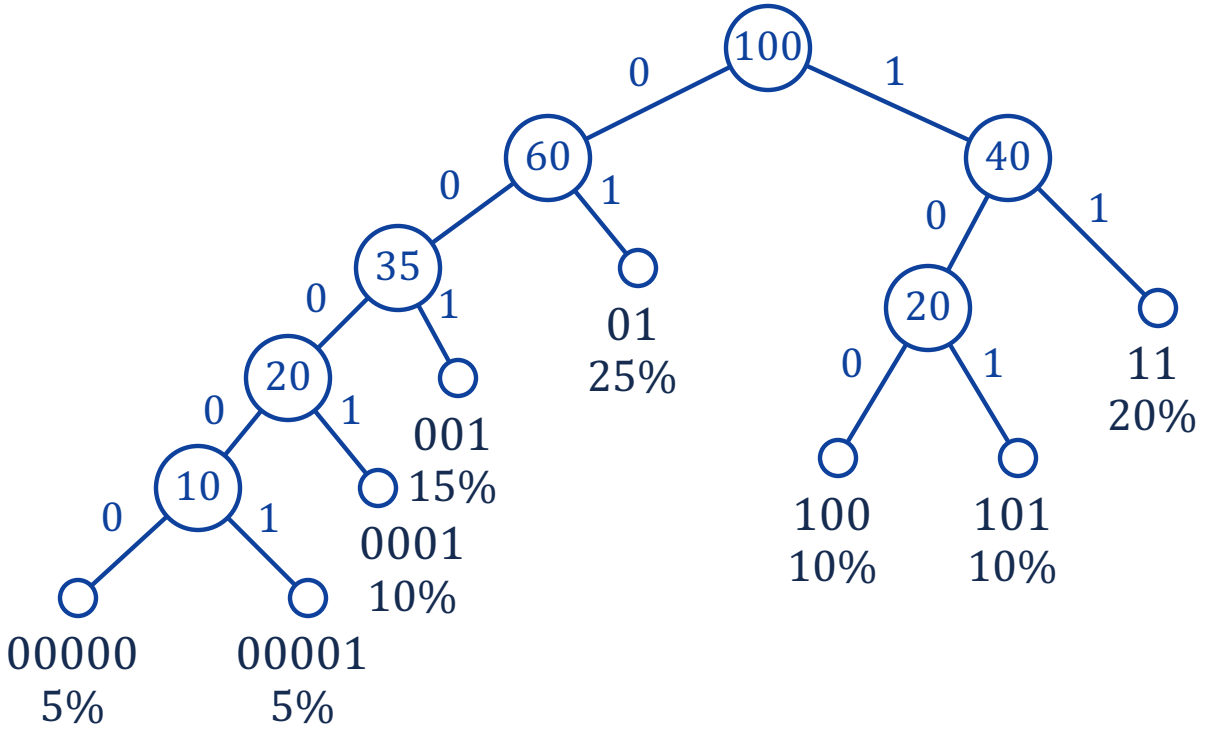
频率总和为100%，码长记为$d(x_i)$。

码字	频率	码字	频率
00000	5%	01	25%
00001	5%	100	10%
0001	10%	101	10%
001	15%	11	20%

$$B = \sum_{i=1}^{n} f(x_i)d(x_i)$$

最优前缀码使 B 最小。

$$B = (5\% + 5\%)\times 5 + 10\%\times 4 + (15\% + 10\% + 10\%)\times 3 + (25\% + 20\%)\times 2 = 2.85$$



最优前缀码问题

输入字符集 $C=\{c_1,c_2,\ldots,c_n\}$ 及每个字符的频率 $f(c_i)$，
求关于 C 的最优前缀码。

Huffman 算法

1. $n \leftarrow |C|$
2. $Q \leftarrow C$，以频率为关键字建立最小优先队列
3. for $i \leftarrow 1$ to $n-1$ do
4. $\quad x \leftarrow \text{ExtractMin}(Q)$；$y \leftarrow \text{ExtractMin}(Q)$
5. $\quad$ 创建节点 z
6. $\quad z.left \leftarrow x$；$z.right \leftarrow y$
7. $\quad f(z) \leftarrow f(x)+f(y)$
8. $\quad \text{Insert}(Q,z)$
9. return $\text{ExtractMin}(Q)$

Huffman 编码

每次取出当前频率最小的两个节点，合并为一个新节点。

新节点的权值等于两个子节点频率之和。

重复此过程，直到只剩一个根节点，得到 Huffman 树。

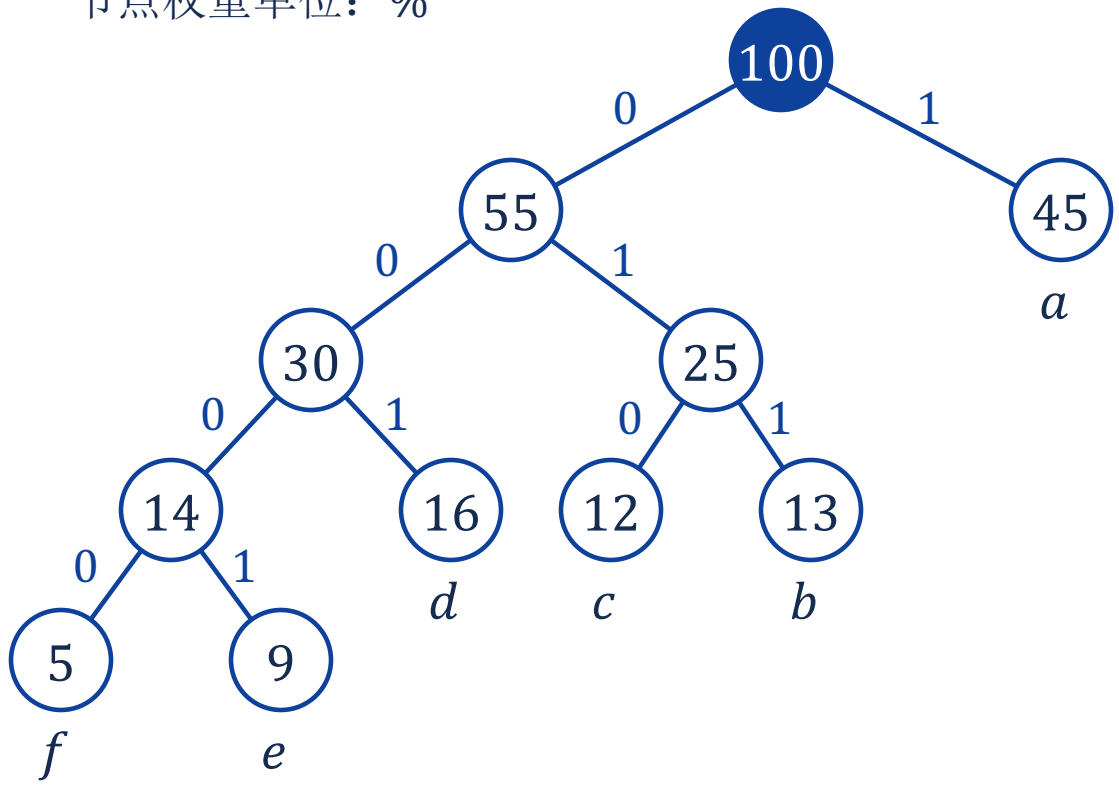
左边标0，右边标1。
从根到叶，沿途的0—1序列就是该字符的二进制码。

Huffman 编码实例

沿树读取完整码字

字符	频率	码字
a	45%	1
b	13%	011
c	12%	010
d	16%	001
e	9%	0001
f	5%	0000

节点权重单位：%



左边标0，右边标1。
从根到叶读取码字。
平均码长：

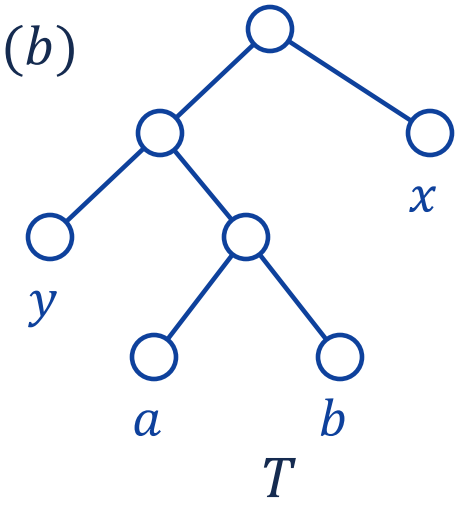
$$4\times(0.05+0.09)+3\times(0.16+0.12+0.13)+1\times0.45=2.24$$

算法分析:引理1，交换

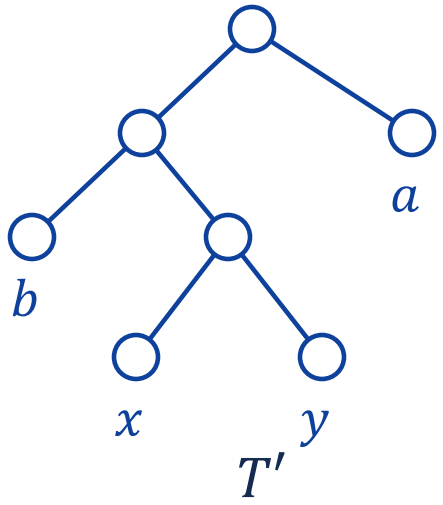
设 x, y 是字符集 C 中频率最小的两个字符。
存在一棵最优前缀码树，使它们为最深的一对兄弟叶子，
因而码字等长，且仅在最后一位不同。
取最优树 T 中最深的兄弟叶子 a, b，交换到 x, y。

$$f(x) \le f(a), \qquad f(y) \le f(b)$$

较小频率移到更深位置，
权值不会增加。
已在目标位置的叶子无需交换。



T



T'

$$B(T) - B(T') = \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \ge 0$$

算法分析:引理2

设 x, y 是前缀码树 T 中的一对兄弟叶子，z 是其父节点。
删除这两个叶子，将 z 作为新叶子，得到 T'。

$$f(z) = f(x) + f(y), \qquad C' = (C \setminus \{x, y\}) \cup \{z\}$$

合并前后的带权路径长度满足：

$$B(T) = B(T') + f(x) + f(y)$$

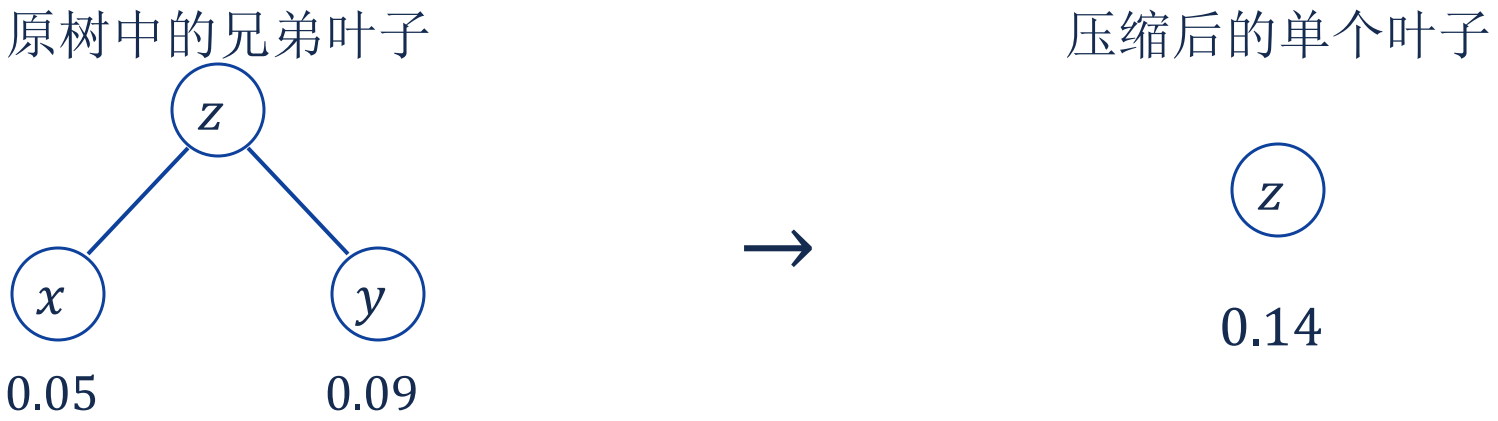


合并节点保存的代价关系

交换引理给出可以合并的最小频率兄弟；代价恒等式说明合并后保留什么。

令 $\ell = d_{T'}(z)$，即父节点 z 的深度。

$$0.05(\ell + 1) + 0.09(\ell + 1) = 0.14\ell + 0.14$$



$$B(T) = B(T') + f(x) + f(y)$$

对固定的一对兄弟，额外代价与剩余树形无关，所以子问题仍可独立优化。

引理2证明

对任意 $c \in C \setminus \{x, y\}$，有 $d_T(c) = d_{T'}(c)$。
且 $d_T(x) = d_T(y) = d_{T'}(z) + 1$。

$$B(T) = \sum_{c \in C \setminus \{x,y\}} f(c)d_T(c) + f(x)d_T(x) + f(y)d_T(y)$$

$$= \sum_{c \in C \setminus \{x,y\}} f(c)d_{T'}(c) + f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1)$$

$$= \sum_{c \in C \setminus \{x,y\}} f(c)d_{T'}(c) + (f(x) + f(y))d_{T'}(z) + f(x) + f(y)$$

$$= (\sum_{c \in C \setminus \{x,y\}} f(c)d_{T'}(c) + f(z)d_{T'}(z)) + f(x) + f(y)$$

$$= B(T') + f(x) + f(y)$$

证明：归纳法

定理：Huffman 算法对任意规模为 $n \geq 2$ 的字符集 C，都得到一棵最优前缀码树。

归纳基础：$n = 2$，字符集 $C = \{x_1, x_2\}$。
算法得到码字0和1，这是最优前缀码。

归纳步骤：假设对任意规模为 k 的字符集，算法都正确。
考虑规模为 $k+1$ 的字符集 $C = \{x_1, x_2, \dots, x_{k+1}\}$，
其中 x_1, x_2 是频率最小的两个字符。

$$C' = (C \setminus \{x_1, x_2\}) \cup \{z\}, \qquad f(z) = f(x_1) + f(x_2)$$

由归纳假设，算法对字符集 C' 及频率 $f(z)$、
$f(x_i)$（$i = 3,4, \dots, k+1$），得到最优前缀码树 T'。

证明：反证法

将 x_1, x_2 作为 z 的子节点附加到 T'，得到树 T。
待证：T 是字符集 $C = (C' \setminus \{z\}) \cup \{x_1, x_2\}$ 的最优前缀码树。

反设 T 不是最优，选取一棵最优树 T^*，满足 $B(T^*) < B(T)$。
由引理1，可选取 x_1, x_2 为最深兄弟叶子的最优树 T^*。

把这两个叶子合并为父节点 z，得到 $T^{*\prime}$。由引理2：

$$B(T^{*\prime}) = B(T^*) - f(x_1) - f(x_2)$$

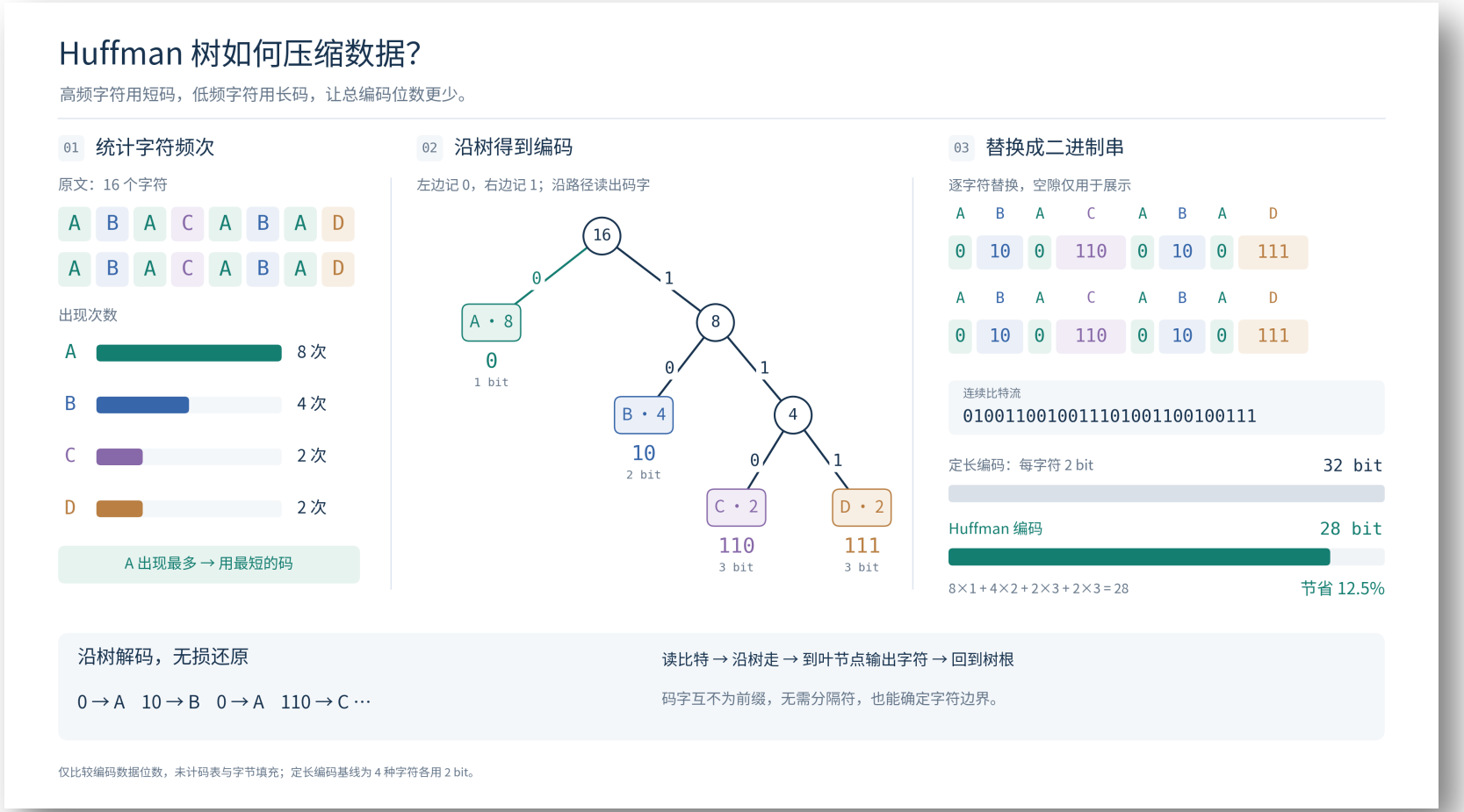
$$< B(T) - f(x_1) - f(x_2) = B(T')$$

这与 T' 是关于 C' 的最优前缀码树矛盾。
因此 T 最优，归纳步骤成立。

Huffman树应用

数据压缩

网络传输优化

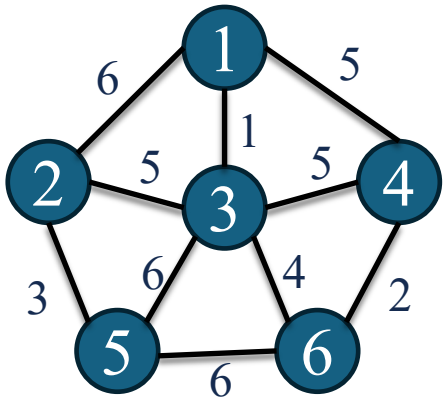


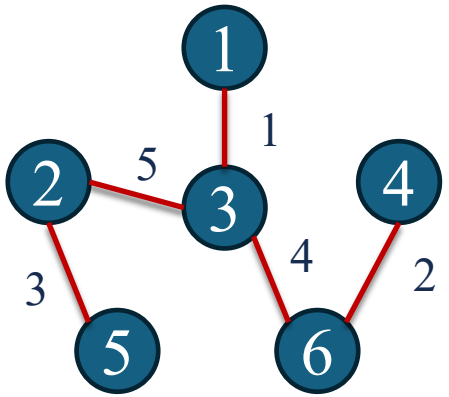
最小生成树 MST

- 给定无向连通带权图 $G = (V, E, W)$。
- 其中 $w(e) \in W$ 是边 e 的权。
- 生成树是边集 $T \subseteq E$，使子图 (V, T) 连通且无回路。
- 树中各边的权之和 $W(T)$ 称为树的权。
- 具有最小权的生成树称为 G 的最小生成树。

实例

- $G=(V,E,W)$，$V=\{1,2,3,4,5,6\}$，边权如下图。
- $E=\{\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,5\},\{3,4\},$
  $\{3,5\},\{3,6\},\{4,6\},\{5,6\}\}$



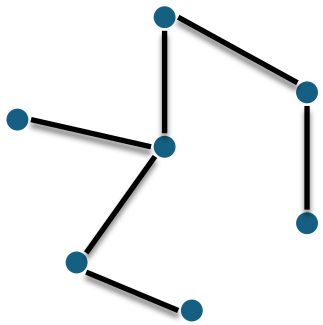


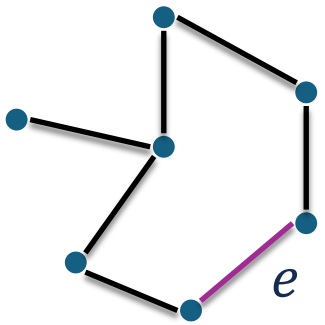
生成树的性质

- 命题 1：设 $G=(V,E)$ 为 n 阶连通图，$T \subseteq E$。

  (V,T) 是 G 的生成树，当且仅当它无圈且有 $n-1$ 条边。

- 若 T 为生成树、$e \in E-T$，则 $T \cup \{e\}$ 含唯一回路 C。

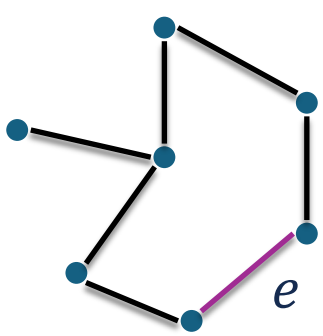


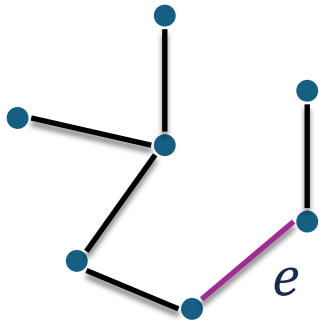


生成树的性质

- 从 $T \cup \{e\}$ 的唯一回路 C 中删去任意一条边，得到 G 的一棵生成树。

  若删去原树中的边，就得到与 T 不同的生成树 T'。





生成树性质的应用

- 算法步骤：逐步选择边。
- 约束条件：不形成回路。
- 终止条件：选中的边数达到 $n-1$。

- 改进生成树 T 的方法：

非树边 e 加入后形成回路 C。删去 C 中的原树边 e_i，得到新树 T'。

$$W(T')-W(T)=w(e)-w(e_i)$$

若 $w(e)\leq w(e_i)$，则 $W(T')\leq W(T)$。

求最小生成树

- 问题：给定无向连通带权图 $G = (V, E, W)$，
  其中 $w(e) \in W$ 是边 e 的权，求 G 的一棵最小生成树。

- 贪心算法：
  Prim 算法
  Kruskal 算法

- 生成树在网络中有重要应用，例如电力系统网络和计算机网络。

割给出的安全边证据

设已选边集 A 能扩展为某棵最小生成树，且割的两边之间没有已选边。



跨割最轻边可安全加入：若最优树未含它，沿形成的环替换另一跨割边。

被替换的边不属于 A；总权值不增加，原先的选择全部保留。

Prim 算法

- 输入：无向连通带权图 $G=(V,E,W)$，$V=\{1,2,\ldots,n\}$。
- 输出：最小生成树 T。
- 设计思想：

  初始 $S=\{1\}$，$T=\emptyset$。

  选择连接 S 与 $V-S$ 的最小权边 $e=\{i,j\}$，其中 $i\in S$、$j\in V-S$。

  将 e 加入 T，将 j 加入 S。

  重复上述过程，直到 $S=V$。

Prim 算法伪码：跨割选择

算法 Prim

1. $S \leftarrow \{1\}$，$T \leftarrow \emptyset$
2. while $S \neq V$ do
3. 　选跨割 $(S, V-S)$ 的最小权边 $e=\{i,j\}$，其中 $i \in S$、$j \notin S$
4. 　$T \leftarrow T \cup \{e\}$；$S \leftarrow S \cup \{j\}$
5. return T

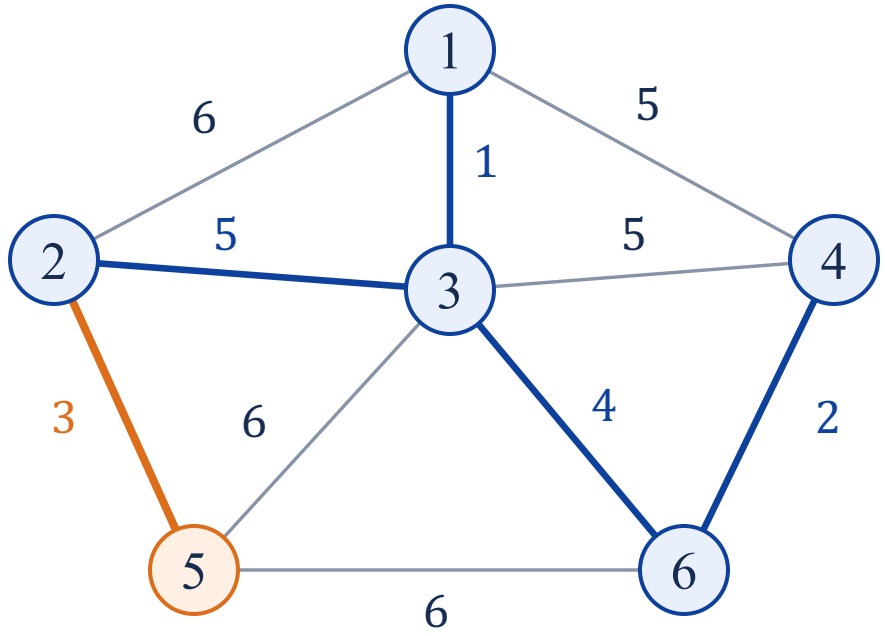
第 5 步：选择 $e=\{2,5\}$，$w(e)=3$

$$S=\{1,2,3,4,5,6\}$$

$$T=\{\{1,3\},\{3,6\},\{6,4\},\{3,2\},\{2,5\}\}$$

依据：当前跨割的最小边权为 3。

跨割选择：每次加入一个新顶点



完成：$|T|=5$，$W(T)=15$。

未选边　已选边　本步选边

正确性证明：归纳法

- 命题：对每个 $1 \leq k \leq n-1$，存在一棵最小生成树包含 Prim 算法前 k 步选出的全部边。
- 归纳基础：$k = 1$。存在一棵最小生成树 T 包含 $e = \{1, i\}$，其中 $\{1, i\}$ 是所有关联顶点 1 的边中权最小的边。
- 归纳步骤：设 $1 \leq k \leq n-2$，并假设存在一棵最小生成树包含前 k 条边；证明存在一棵最小生成树包含前 $k+1$ 条边。

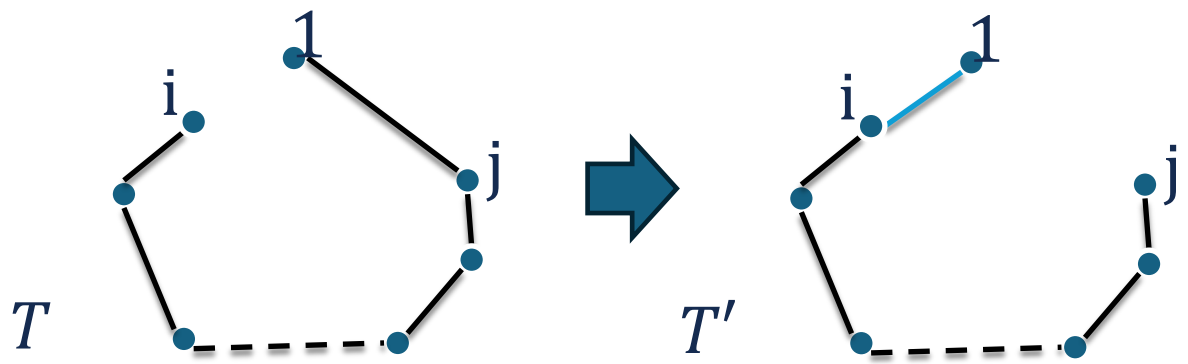
归纳基础

- 存在一棵最小生成树，包含关联顶点 1 的最小权边 $e = \{1, i\}$。
- 证明：设 T 为一棵最小生成树。若 $e \in T$，结论成立。

  否则，$T \cup \{e\}$ 中有唯一回路，其关联顶点 1 的另一条边为 $\{1, j\}$。

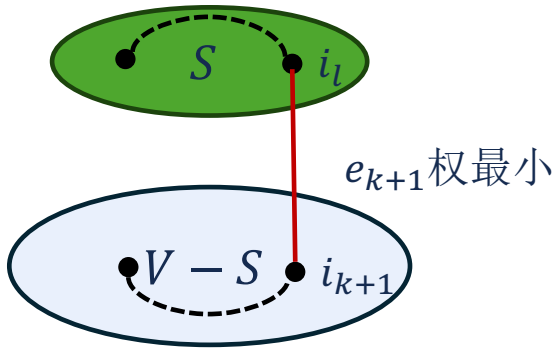
  用 e 替换 $\{1, j\}$ 得到生成树 T'；由 $w(e) \leq w(\{1, j\})$，有 $W(T') \leq W(T)$。

  因 T 已最优，必有 $W(T') = W(T)$，故 T' 也是最小生成树，且包含 e。



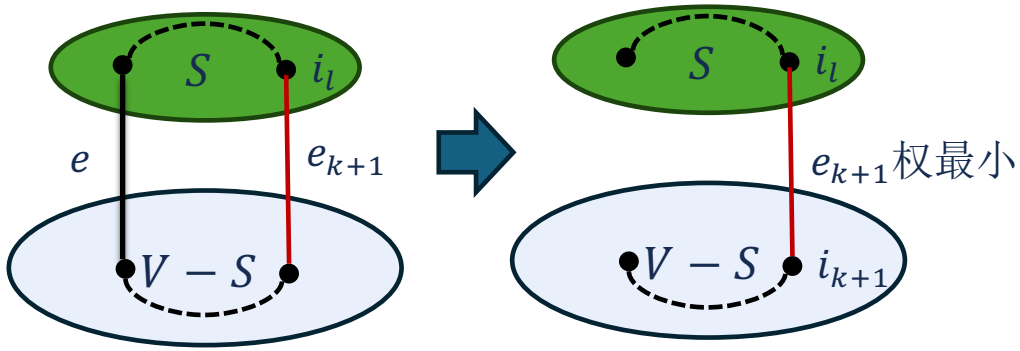
归纳步骤

- 前 k 步选出的边为 $e_1, e_2, \ldots, e_k$，它们连同起点形成顶点集 S。
  因为选中的边构成一棵树，所以 $|S| = k + 1$。
- 由归纳假设，存在 G 的一棵最小生成树 T 包含这些边。
- 第 $k + 1$ 步选取跨割 $(S, V - S)$ 的最小权边 $e_{k+1} = \{i_l, i_{k+1}\}$，
  其中 $i_l \in S$、$i_{k+1} \notin S$。若 $e_{k+1} \in T$，归纳不变式继续成立。



归纳步骤

- 若 $e_{k+1} \notin T$，将其加入 T 形成唯一回路；回路中另有一条跨割 $(S, V-S)$ 的边 e。前 k 条边都在 S 内，因此删去 e 不会删去这些边。
- 令 $T^* = (T - \{e\}) \cup \{e_{k+1}\}$，则 T^* 仍为生成树，包含前 $k+1$ 条边。
  由 $w(e_{k+1}) \leq w(e)$，有 $W(T^*) \leq W(T)$；因 T 已最优，T^* 也是最小生成树。
- 所以归纳不变式成立；当 $k+1 = n-1$ 时，算法得到最小生成树。



时间复杂度

- 采用数组实现：维护每个未选顶点到 S 的最小连接权。
- 每轮选择一个顶点，共执行 $n-1$ 轮，即 $O(n)$ 轮。
- 每轮的工作：

  扫描数组，找到最小连接权及对应边：$O(n)$。

  新顶点加入 S 后，更新其余顶点的最小连接权：$O(n)$。
- 总时间复杂度：$O(n^2)$。

Kruskal 算法

- 输入：无向连通带权图 $G=(V,E,W)$，$V=\{1,2,\ldots,n\}$。
- 输出：最小生成树 T。
- 设计思想：

  初始 $T=\emptyset$，按边权非降序排列所有边。

  依次考察边 e：若加入 e 不会与 T 中的边构成回路，

  则将 e 加入 T；否则跳过。

  直到选出 $n-1$ 条边为止。

Kruskal 算法伪码：合并连通分量

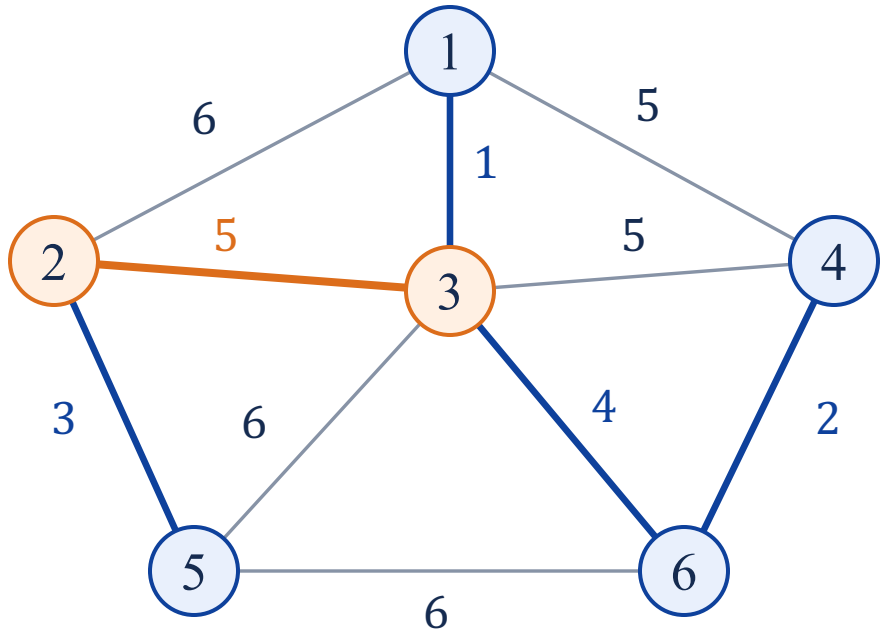
算法 Kruskal
输入：无向连通带权图 $G=(V,E,W)$
顶点数 n、边数 m；输出：最小生成树 T。

1. 按边权非降序排列 E
2. $T \leftarrow \emptyset$；为每个顶点建立
   一个单元素集合
3. 按排序顺序依次考察边 $e=\{u,v\}$
4. 　if find$(u) \neq$ find(v) then
5. 　　$T \leftarrow T \cup \{e\}$
6. 　　union(u,v)
7. 　if $|T| = n-1$ then break
8. return T

$$T = \{\{1,3\},\{4,6\},\{2,5\},\{3,6\},\{2,3\}\}$$

已选边数：5/5；当前权值和：15。

按边权考察：1,2,3,4,5, …



第 5 步：选 $e=\{2,3\}$，$w(e)=5$
同权边中先取 $\{2,3\}$；生成树完成。

未选边　已选边　本步选边

步骤 6/6 · 第5步：选边{2,3}，边权5

正确性证明思路

- 命题：对任意 $n \geq 1$ 的无向连通带权图，Kruskal 算法得到一棵最小生成树。归纳中允许平行边，并忽略自环。

- 证明思路：对顶点数作数学归纳。

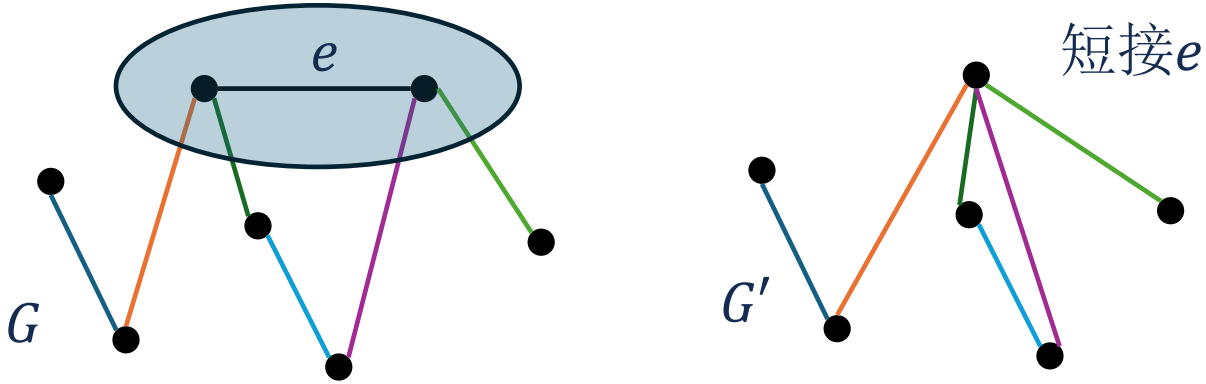
  $n = 1$ 时，空边集即为最小生成树。

  $n = 2$ 时，选择两顶点之间最小权的边，即为最小生成树。

- 归纳步骤：假设算法对 n 阶图正确，证明对任意 $n + 1$ 阶无向连通带权图也能得到最小生成树。

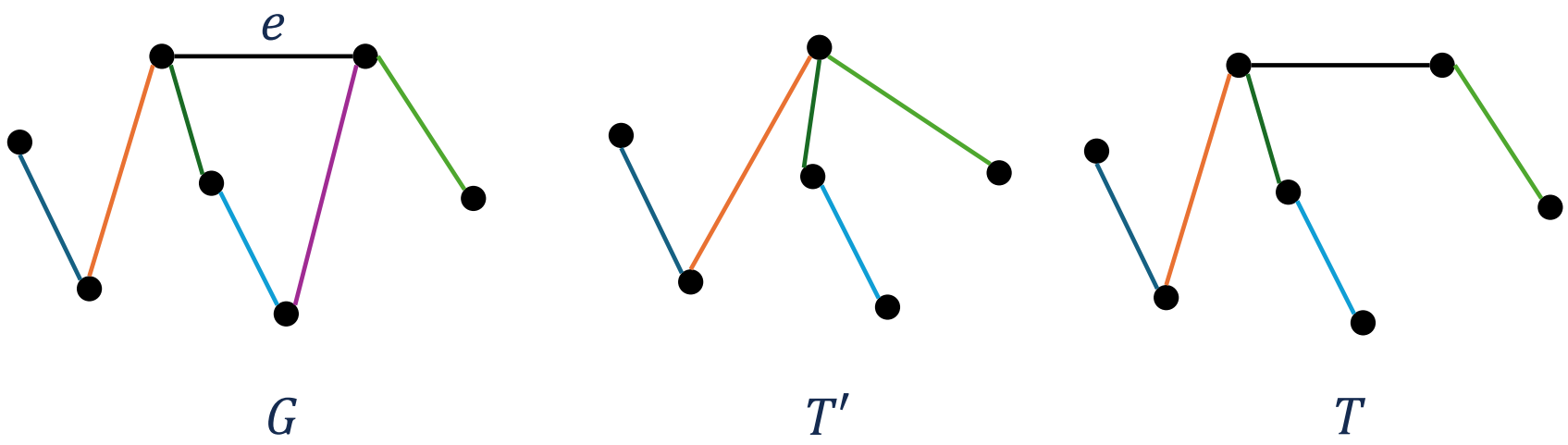
短接操作

- 设 G 为 $n+1$ 阶无向连通带权图，$e=\{i,j\}$ 是全图最小权边。
- 收缩（短接）e：合并顶点 i,j，得到有 n 个顶点的图 G'。
- 删除自环；保留平行边、原边权及原边对应关系。



归纳步骤证明

- 对任意 $n+1$ 阶无向连通带权图 G，收缩全图最小权边 e，得到 n 阶图 G'。
- 固定同权边的先后顺序。选出 e 后，剩余步骤等价于在 G' 上运行 Kruskal。
- 根据归纳假设，算法得到 G' 的最小生成树 T'。
- 把 T' 的边映射回原图，再加入 e，得到生成树 $T = T' \cup \{e\}$。
- 下面证明：T 是 G 的最小生成树。



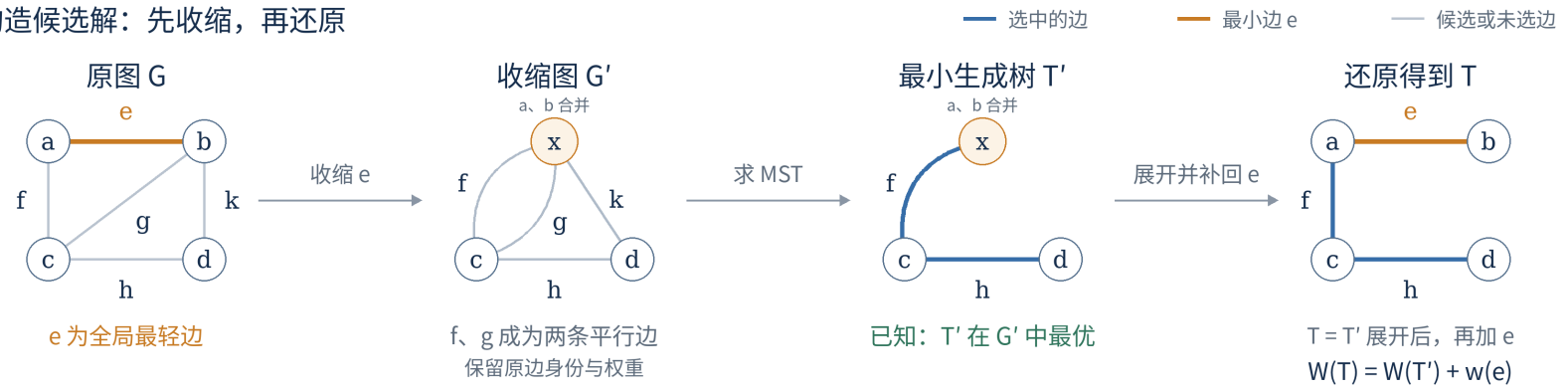
证明

- 设 $T = T' \cup \{e\}$，其中 T' 是收缩图 G' 的最小生成树，
  且 $W(T) = W(T') + w(e)$。反设存在 G 的最小生成树 T^*，满足 $W(T^*) < W(T)$。
- 若 $e \notin T^*$，在 T^* 中加入 e 得唯一回路 C，任取 $f \in C - \{e\}$。
  因 e 为全图最小权边，$w(e) \leq w(f)$。令 $U = (T^* - \{f\}) \cup \{e\}$，则 U 仍为生成树。
  $W(U) \leq W(T^*)$；由 T^* 的最优性知二者相等，故 U 也是最小生成树且包含 e。
- 若 $e \in T^*$，直接令 $U = T^*$。
- 在 U 中收缩 e，得到 G' 的生成树 U'；其边对应 $U - \{e\}$，且
  $W(U') = W(U) - w(e) = W(T^*) - w(e) < W(T) - w(e) = W(T')$。
- 这与 T' 是 G' 的最小生成树矛盾。因此 T 为 G 的最小生成树。

一张图看懂：最小边收缩的正确性证明

前提：G 为连通无向带权图；e 是全图最小权边。收缩 e：合并 a、b 为 x，并删去 e。

A 构造候选解：先收缩，再还原



e 为全局最轻边

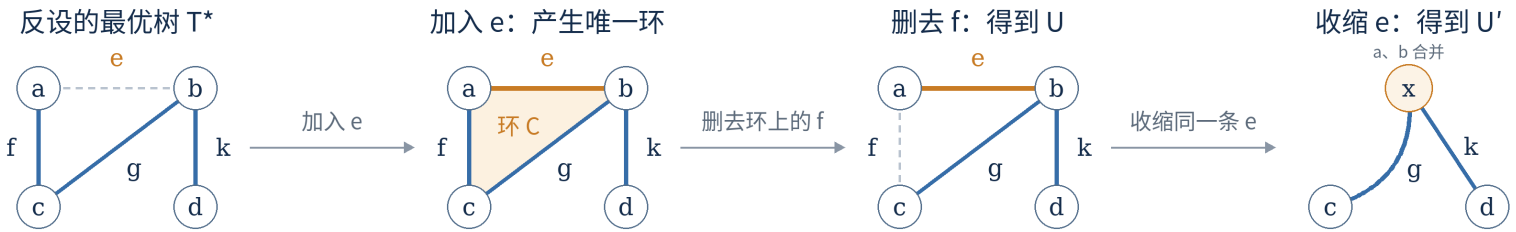
f、g 成为两条平行边
保留原边身份与权重

已知：T′ 在 G′ 中最优

T = T′ 展开后，再加 e
W(T) = W(T′) + w(e)

B 反设有更轻的最优树 T*：W(T*) < W(T)

下排画 e ∉ T* 的情形；若 e ∈ T*，直接令 U = T*，进入最后的收缩与比较。



假设：W(T*) < W(T)

e 最轻，所以 w(e) ⩽ w(f)

换边不增重；T* 已最优
所以 W(U) = W(T*)

U′ 是 G′ 的一棵生成树
W(U′) = W(T*) − w(e)

C 比较：两棵树都减去同一个 w(e)，轻重关系不变

$$W(U') = W(T^*) - w(e) \;<\; W(T) - w(e) = W(T')$$

U′ 竟比已知最优的 T′ 更轻 —— 矛盾！

因此，T 是 G 的最小生成树。

两种生成树算法的不变式

共同不变式：存在一棵最小生成树包含当前已选边集 A。

算法	当前结构	安全边的来源
Prim	已选边构成连接 S 的树	割 $(S, V-S)$ 上的最轻边
Kruskal	已选边构成森林	按权扫描、连接不同块的边

Kruskal接受的边，是加入前某个连通块对外的最轻边之一。

判环保证加入后仍为森林；权值顺序与割性质共同保证仍可扩展为最优树。

Kruskal 做了什么

- 排序：把 m 条边按边权非降序排列。
- 扫描：按顺序依次考察每条边。
- 合并：若两端属于不同集合，就加边并合并集合（Union-Find）。

$\text{find}(x)$：查找顶点 x 所在连通块的代表，查找时做路径压缩。

$\text{union}(x, y)$：合并两个连通块；可按大小合并，将小树的根接到大树的根。

复杂度分析

- 排序 m 条边：使用归并排序或堆排序等，时间为 $O(m \log m)$。
- 这是所选排序算法的上界，并非所有比较排序都具有这一上界。
- 对稠密简单图，$m = \Theta(n^2)$，排序时间为 $O(n^2 \log n)$。

并查集的代价

- 在 Kruskal 中，每条边只需常数次 find；若复用代表根，判环只需 2 次。

  成环时不合并；被接受的边有 $n-1$ 条，因此执行 $n-1$ 次 union。

- 使用按大小合并与路径压缩：初始化及全部并查集操作共需 $O((n+m)\alpha(n))$；对连通图，可写为 $O(m\alpha(n))$。

- $\alpha(n)$ 是增长极慢的反 Ackermann 函数，因此该开销接近线性。

- 加上排序，总时间复杂度为 $O(m \log m)$。

生成代码仍需保持算法契约

数学算法得到正确性证明后，实现仍必须保持同一组条件与状态更新。

层次	活动选择中的具体要求
规格	半开区间允许 $f_i = s_j$ 的两项活动衔接
算法与实现	接纳条件保留 $s_i \geq f_j$，接受后才更新 j
最小边界实例	$[0,1), [1,2)$ 可选两项；误用 $>$ 只能选一项

HumanEval用执行单元测试评价生成程序的功能正确性（Chen等，2021）。

测试能发现反例；全输入的保证还需要算法证明，以及实现对规格的保持。
验收AI生成的候选代码，也应逐项核对这些条件。

贪心法的局限性

- 贪心策略：每一步做局部最优选择。
- 局限：局部最优选择不保证得到全局最优解。
- 可能的原因：

  问题不具备所采用策略需要的贪心选择性质；

  当前的局部决策会影响后续选择的可行性；

  局部选择未必能通过交换论证等方法证明可扩展为全局最优解。

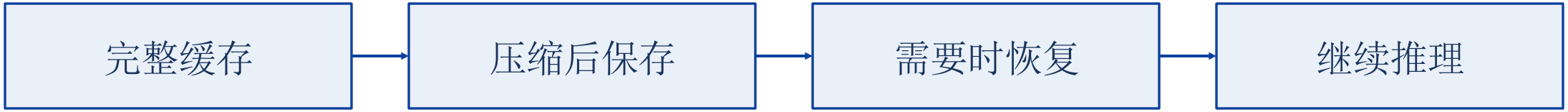
长文对话中的历史计算



缓存保存什么	直观作用
键 K：历史位置的数值表征	帮助匹配当前应关注的位置
值 V：历史内容的数值表征	提供注意力计算要汇总的信息

KVTC：压缩保存缓存

如何处理同一份缓存	得到什么	付出什么
完整保存	随时复用	持续占空间
直接丢弃	释放空间	下次可能重新计算
压缩保存	减少占用并保留复用机会	编解码时间与近似误差



KVTC 的目标：在存储预算内，尽量保留有用的历史计算。

PCA：给缓存换一组坐标

数值之间存在相关性：有些列经常一起变化



直觉：两列经常一起升降，可以主要记录共同变化，再记录差异。

PCA 即主成分分析，用代表性数据学习这样的坐标方向。

保留全部分量时可逆变换；量化或省略分量会引入近似误差。

量化：用有限档位记录数值

同一分量中的一个数：1.40

记录精度	允许的恢复档位（示例）	恢复值
0 bit	省略该分量	0
2 bit	0、0.5、1、1.5，共4档	1.5
4 bit	0、0.1、…、1.5，共16档	1.4

位数越多，可用档位越细；存储成本也越高。

每个位置的六比特怎样分配

每个位置预算6 bit；A、B、C各可选0、2、4 bit

分量	0 bit	2 bit	4 bit
A	12	9	0
B	10	4	3
C	8	3	2

平均分配
A、B、C 各 2 bit

预算用满 6 bit
总误差为 16

平均分配只是一个可行方案，接下来系统比较其他分法。

动态规划先保留不同预算的最佳结果

按分量逐步计算，每一列对应一种可用预算上限

已处理分量	预算0	预算2	预算4	预算6
只有 A	12	9	0	0
A 和 B	22	16	10	4

右下角这一格：比较 B 的三种精度

B 使用的位数	0 bit	2 bit	4 bit
留给 A 的预算	6 bit	4 bit	2 bit
两者总误差	10	4（最小）	12

加入 C，再读出完整分配方案

C 的位数	留给 A、B	前两项误差	C 的误差	总误差
0 bit	6 bit	4	8	12
2 bit	4 bit	10	3	13
4 bit	2 bit	16	2	18

A 用 4 bit

B 用 2 bit

C 用 0 bit

总误差 12；真实 KVTC 还同时决定分组，并计入组参数开销。

同一例子的贪心分配对照

示例规则：每次加 2 bit，选择本次误差下降最多的分量



方法	A、B、C 的位数	总误差
上述贪心规则	2、2、2	16
动态规划	4、2、0	12

A 的两次追加投入分别改善 3、9：后续收益依赖前面的投入。

无损编码中的 Huffman 贪心



组件	利用什么规律	与本讲的联系
LZ77	重复出现的片段	用已有片段的引用减少重复
Huffman	符号出现频率不同	经典贪心：反复合并最小频率节点

无损解码恢复量化字节；量化产生的近似误差仍然存在。

KVTC 的准备、压缩与恢复

准备：代表性数据确定 PCA 坐标；DP 按预算选择分组与精度

保存时



复用时



需要同时付出参数存储、变换与恢复时间，并验证目标任务的质量。

实验：空间节省与回答质量

配置名（目标档位）	压缩部分的压缩率	数学题 GSM8K	长文检索 LITM
原始缓存	1 倍	56.8	99.4
KVTC 16×	18 - 22 倍	56.9	99.3
KVTC 64×	60 - 88 倍	57.2	90.2

中等压缩接近基线；更强压缩可能损伤长文检索。

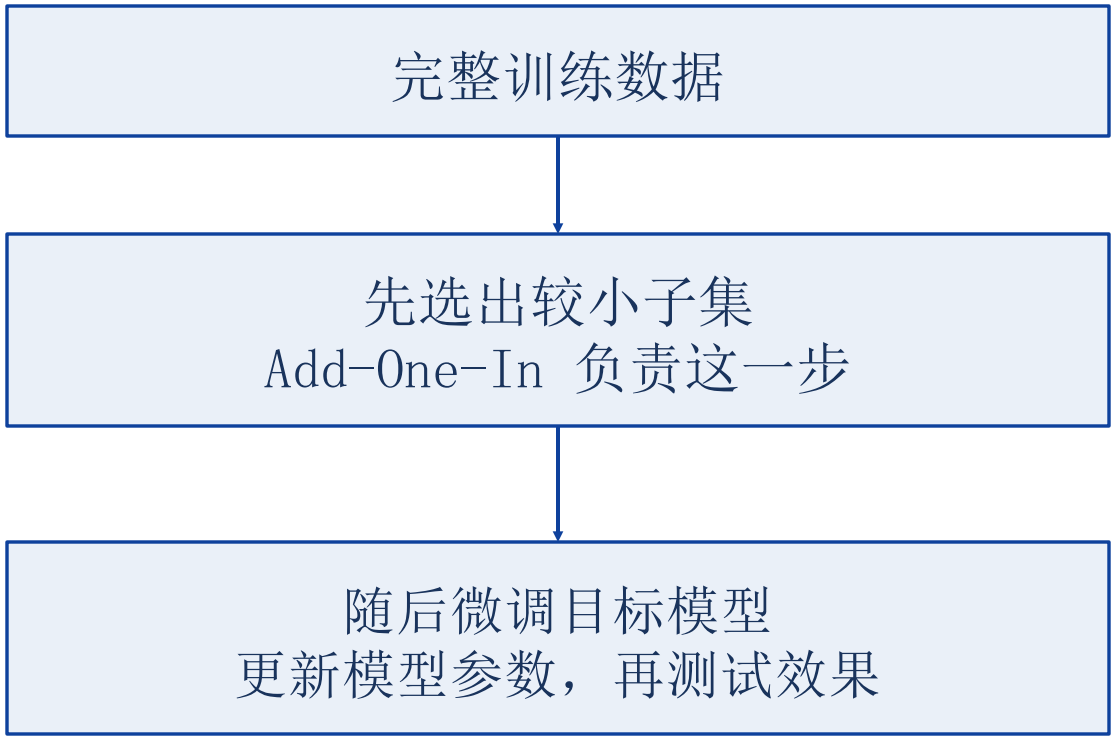
16×、64×是配置名，实际压缩率见第二列。

未计原精度保留的缓存；还需计入参数与编解码成本。

微调与一条训练样本

一条样本 a
指令：把输入数字求和
输入：2、3、5
附带答案：10

选择模型负责比较候选。
目标模型随后学习选出的样本。



单独高分的样本仍可能重复

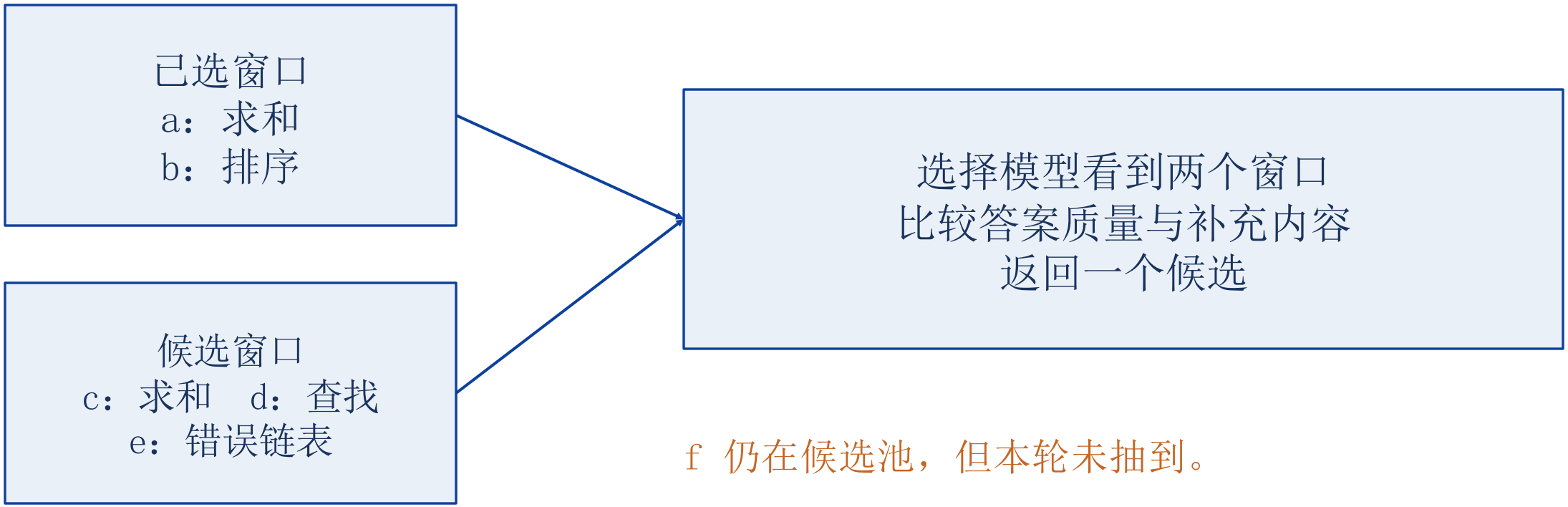
编号	指令与输入	数据中附带的答案
a	求和：2、3、5	10
b	升序排列：3、1、2	1、2、3
c	求和：4、6	10
d	在有序表 1、4、7 中查找 4	第2个位置
e	反转链表 1、2、3	3、1（漏掉节点2）
f	从 A 遍历相邻节点 B、C	A、B、C

假设独立评分取前4条：a、b、c、d。a 与 c 重复，f 未入选。

目标仍是选4条；选择下一条时，需要看看已经选了什么。

每轮交给选择模型的两个窗口

随机起点：已选 a、b；候选池：c、d、e、f；目标：4条



f 仍在候选池，但本轮未抽到。

第一轮：选择 d 并更新集合

本轮已选窗口：a（求和）、b（排序）

候选	回答质量	相对已选内容的贡献
c：求和	正确	与 a 重复
d：二分查找	正确	补充查找任务
e：链表反转	漏掉节点2	类别新，但回答有错

选择 d 后
已选：a、b、d，共3条

候选池只移走 d
剩下：c、e、f

第二轮：选择 f，选满四条停止

完整已选仍是 a、b、d；本轮只抽出 b、d 给模型看

已选窗口
b：排序　d：二分查找

候选窗口
c：求和　e：错误链表
f：正确图遍历

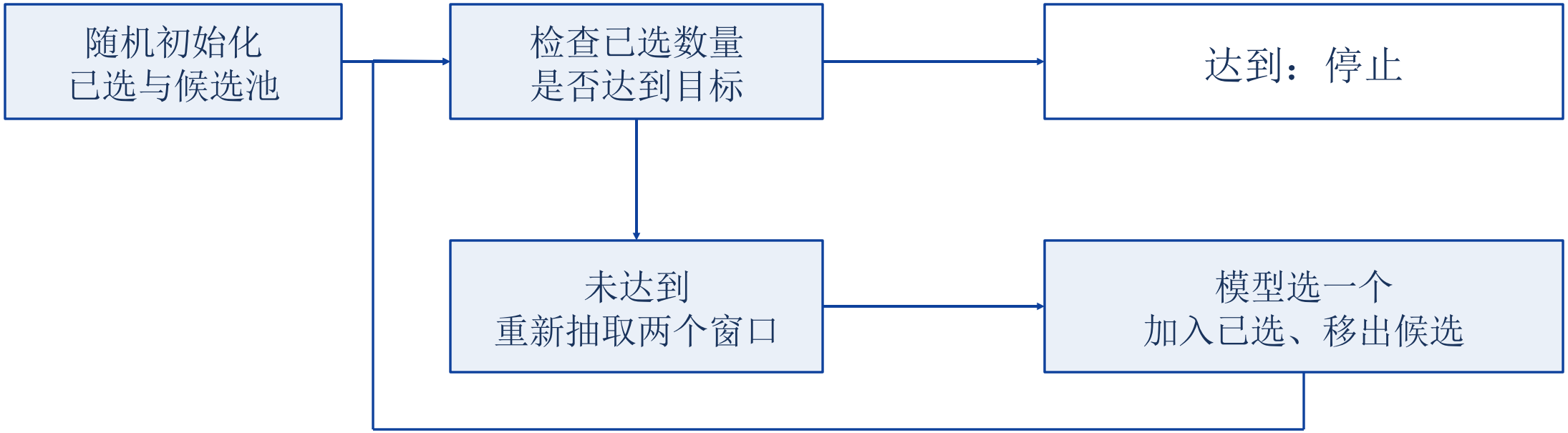
结合两个窗口
示例选择 f，补充图遍历

a 未被删除；f 在本轮选择前仍在候选池。

最终已选：a、b、d、f，共4条。达到目标，停止。

Add-One-In 的完整贪心流程

每轮比较加入一条的新增价值，称为“边际贡献”



论文默认：种子20条，两个窗口各20条。教学例子用2条和3条。

局部信息与模型判断的边界

可能发生的情况	同一例子里的表现	影响
没看到有价值的候选	第一轮没有抽到 f	本轮无法选它
没看到已有的相似内容	第二轮没有展示 a	可能低估 c 的重复
模型评价出错	没有发现 e 漏掉节点2	可能选入错误答案

扩大窗口可以增加可见信息，也会增加提示长度与判断成本。

实际效果需要验证，不能直接承诺选出全局最优的训练集。

实验：少量样本能否支持有效微调

先选9千条，再微调目标模型；约为全量5.2万条的17.3%

微调后的测试任务	全量52K	Add-One-In 9K
知识测试 MMLU	45.45	47.44
常识情境 HellaSwag	69.97	71.07
复杂推理 BBH	32.10	30.97

有的任务上升，有的下降；选样阶段本身也需要计算。

资源限制与算法选择

算法所在环节	每次决定什么	资源与目标	为什么这样做
KVTC 量化配置	分组与精度	有限比特内 减小重构误差	DP 保留不同预算 下的最佳组合
KVTC 无损编码	符号与码长	按频率缩短编码 精确保留量化字节	Huffman 经典贪心 合并最小频率节点
Add-One-In 选样	下一条训练样本	选满目标数量 兼顾质量与补充内容	结合已选背景 逐轮做局部判断

同样面对资源限制，问题结构和可获得的信息决定算法策略。